



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практическим работам №9-12

по дисциплине «Технологические основы Интернета вещей»

Выполнили:

Студенты группы ИКБО-42-23

Голев С.С.
Мареева А.Р.
Кунин А.И.

Проверил:

Ибатулин М.Ю.

Москва 2025г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	2
ПРАКТИЧЕСКАЯ РАБОТА №9.....	4
2.1 Цель работы.....	4
2.2 Постановка задачи.....	4
2.3 Ход работы.....	4
2.4 Заключение.....	9
ПРАКТИЧЕСКАЯ РАБОТА №10.....	10
3.1 Цель работы.....	10
3.2 Постановка задачи.....	10
3.3 Ход работы.....	10
3.4 Заключение.....	16
ПРАКТИЧЕСКАЯ РАБОТА №11.....	18
4.1 Цель работы.....	18
4.2 Постановка задачи.....	18
4.3 Ход работы.....	18
ПРАКТИЧЕСКАЯ РАБОТА №12.....	23
5.1 Цель работы.....	23
5.2 Постановка задачи.....	23
5.3 Ход работы.....	23
5.4 Заключение.....	28
ЗАКЛЮЧЕНИЕ.....	29
СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ.....	30

ВВЕДЕНИЕ

Современные платформы Интернета вещей (IoT) позволяют не только управлять устройствами и собирать телеметрию, но и формировать сложную логику их поведения, обеспечивая автоматизацию процессов и мониторинг состояния систем в реальном времени. Одним из ключевых элементов такой архитектуры является модель объекта контроля — формализованное описание устройства, включающее его параметры, команды и возможные события. Благодаря моделям обеспечивается унификация работы с различными устройствами, снижение зависимости от протоколов передачи данных и повышение удобства разработки IoT-решений.

В рамках данной практической работы рассматриваются базовые принципы построения моделей объектов на облачной платформе Rightech IoT Cloud, подготовка их структуры, создание аргументов, действий, событий и конфигурационных параметров. Также изучаются возможности платформы по построению сценариев автоматизации с использованием конечных автоматов, позволяющих задавать последовательность состояний устройства и правила переходов между ними на основе входящих данных.

Отдельное внимание уделяется механизму сообщений и тревог, применяемому для отслеживания состояния физических устройств и своевременного реагирования на возникающие отклонения. В работе исследуется процесс формирования уведомлений, их визуализация в интерфейсе платформы, а также расширенные методы оповещения через внешние сервисы, включая SMTP-email и другие каналы.

Цель практической работы заключается в формировании практического понимания построения моделей устройств, создания автоматизации и систем предупреждений на платформе Rightech, а также освоении инструментов контроля, позволяющих создавать полноценные IoT-сервисы с возможностью мониторинга, анализа и реакции на события в системе.

ПРАКТИЧЕСКАЯ РАБОТА №9

2.1 Цель работы

Цель: подключить и настроить устройство с поддержкой MQTT на платформе Rightech IoT Cloud, используя внешний MQTT-брокер.

2.2 Постановка задачи

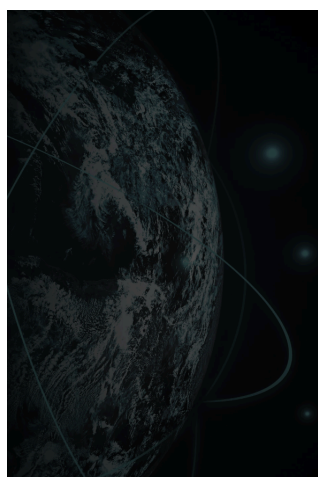
1. Регистрация на платформе Rightech IoT Cloud.
2. Создание модели устройства.
3. Настройка MQTT брокера.
4. Добавление устройства в облако Rightech.
5. Настройка физического устройства.
6. Мониторинг данных на платформе.
7. Тестирование и отладка.

2.3 Ход работы

Часть 1. Регистрация на платформе Rightech IoT Cloud:

Rightech IoT Cloud — это бескодовая (no-code) IoT-платформа для быстрого создания прикладных проектов интернета вещей.

Программный продукт Rightech IoT Cloud (RIC, рус. Райтек IoT Клауд) от компании разработчика КОМНЭТ является фреймворк-инструментом и предназначен для быстрого создания разработчиками приложений интернета вещей) Платформа RIC реализована на принципах универсализации и имеет низкую зависимость от конкретного оборудования и протоколов, что позволяет легко объединять разнородные устройства в едином разрабатываемом на платформе решении. Создадим на данной платформа собственный аккаунт (рис. 2.1).



© Rightech 2016-2025

Рисунок 2.1 – Регистрация на платформе Rightech IoT Cloud

Часть 2. Создание виртуальных устройств в облаке

Согласно варианту по номеру бригады создадим в облаке виртуальные устройства для получения данных(табл. 2.1). Каждое устройство должно иметь свой профиль, соответствующий передаваемым на устройство данным. В качестве протокола для профилей устройств используем MQTT.

Таблица 2.1 - индивидуальный вариант

№ варианта	Датчики
5	1. Датчик качества воздуха 2. Датчик освещенности 3. Датчик напряжения

Выполним передачу тестовых данных в каждое из созданных устройств. Для начала создадим три модели для наших устройств: voltage, CO₂, illum. Зададим им аргументы(параметр, который передает устройство на платформу, например текущее измерение сенсора).

Далее введем уникальный по модели идентификатор и имя узла. Идентификатор необходим для распознавания узла в системе. Он является названием поля из данных о состоянии устройства при обращении к нему через API. Имя, в отличие от идентификатора, может быть неуникальным. Как идентификатор и как имя для удобства будем использовать названия датчиков.

Тип данных будем использовать один общий - число(number). Единицы измерения: датчик качества воздуха - проценты(%), датчик напряжения - ватты(Вт) и датчик освещенности оставим по умолчанию.

Согласно описанным параметрам заполним датчик CO2 (рис. 2.2).

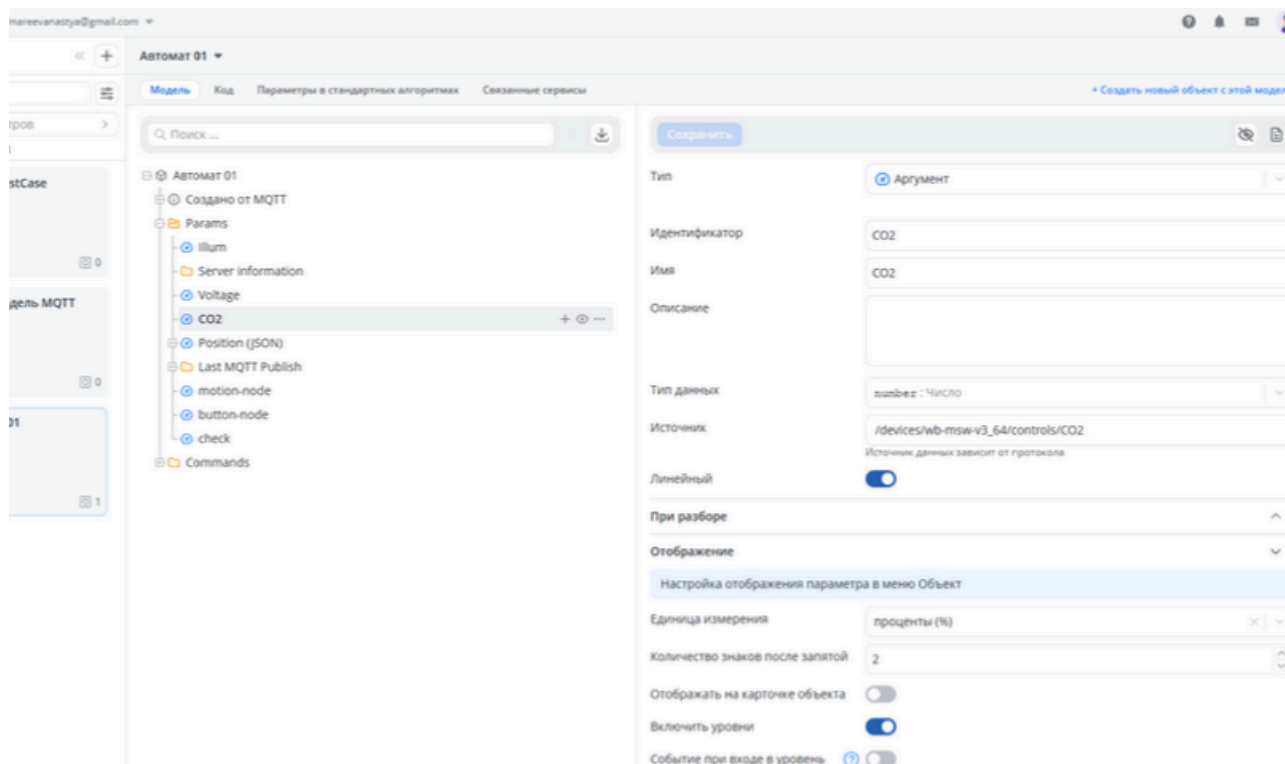


Рисунок 2.4 – Автомат illum

Согласно описанным параметрам заполним датчик Voltage (рис. 2.3).

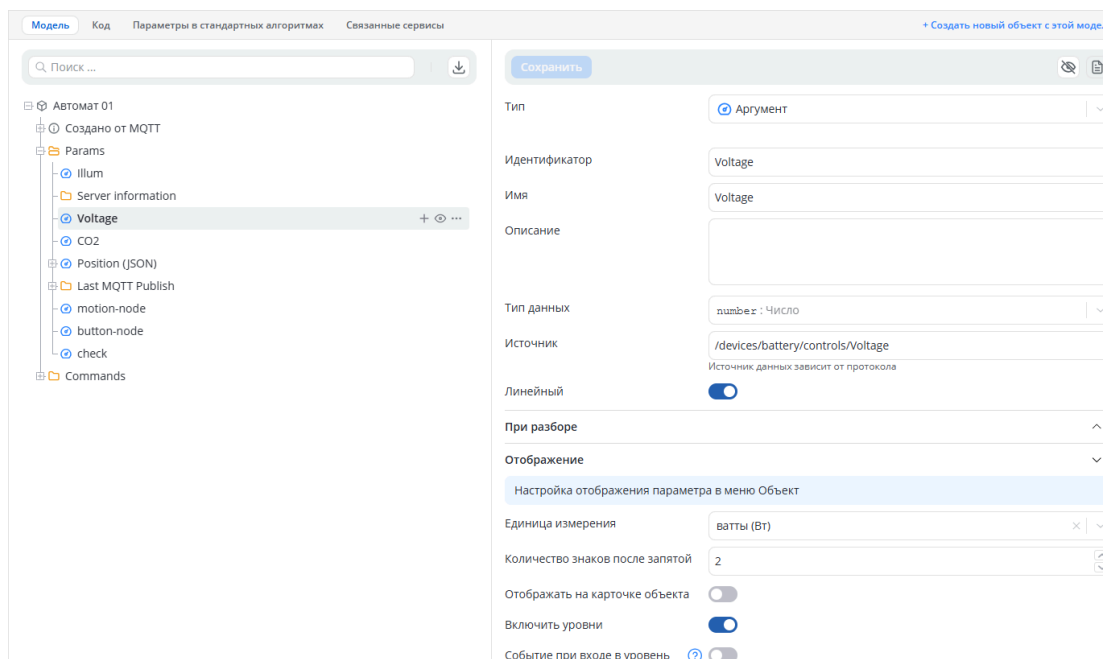


Рисунок 2.3 – Автомат Voltage

Согласно описанным параметрам заполним датчик illum (рис. 2.4).

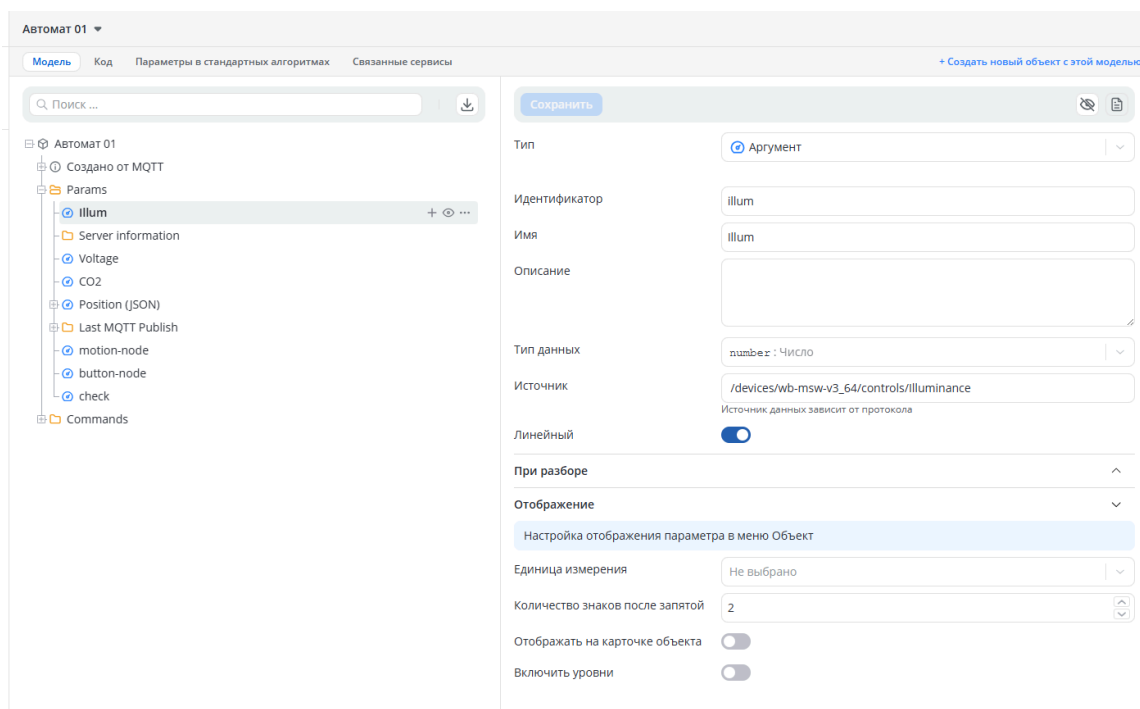


Рисунок 2.4 – Автомат illum

Создадим аргумент с типом данных Объект(рис 2.5), в источнике укажем тот топик, по которому ожидаем данные - в данном случае base/state/pos. Зададим некоторый идентификатор - например, pos.

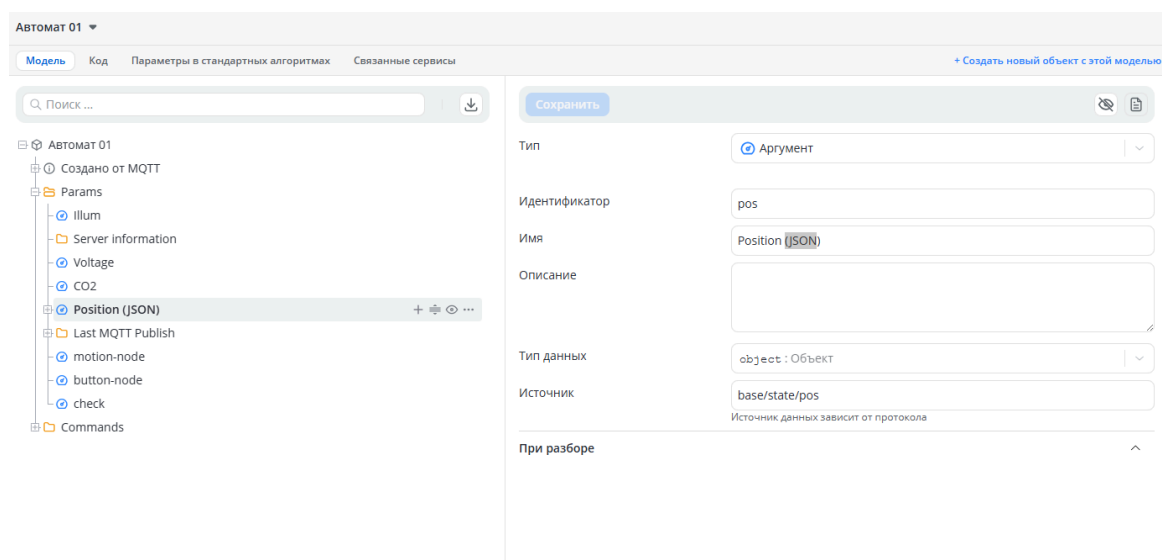


Рисунок 2.5 – Аргумент с типом данных Объект

Часть 3. Отправка данных в облако

В разделе «Объекты» на платформе Rightech добавьте новое устройство. Выберем ранее созданную модель и укажите параметры подключения: адрес MQTT брокера, порт, топики для подписки и публикации данных.

Далее отправим сообщения по протоколу MQTT на созданные топики(рис. 2.8).

```
cert : path to a file containing private key and certificate for encrypted
communication.
--capath : path to a directory containing trusted CA certificates to enable encrypted
communication.
--cert : client certificate for authentication, if required by server.
--key : client private key for authentication, if required by server.
--keyform : keyfile type, can be either "pem" or "engine".
--ciphers : openssl compatible list of TLS ciphers to support.
--tls-version : TLS protocol version, can be one of tlsv1.3 tlsv1.2 or tlsv1.1.
Defaults to tlsv1.2 if available.
--insecure : do not check that the server certificate hostname matches the remote
hostname. Using this option means that you cannot be sure that the
remote host is the server you wish to connect to and so is insecure.
Do not use this option in a production environment.
--tls-engine : If set, enables the use of a TLS engine device.
--tls-engine-kpass-sha1 : SHA1 of the key password to be used with the selected SSL engine.
--tls-use-os-certs : load and trust OS provided CA certificates.
--psk : pre-shared-key in hexadecimal (no leading 0x) to enable TLS-PSK mode.
--psk-identity : client identity string for TLS-PSK mode.
--proxy : SOCKS proxy URL of the form:
socks://[username[:password]@]hostname[:port]
Only "none" and "username" authentication is supported.

See https://mosquitto.org/ for more information.

user@fireboard-ASV7IXPS:~$ mosquitto_pub -d -h dev.rigtech.io -i mqtt-mareevanastya-d10vxo -t "/devices/wb-msw-v3_64/controls/CO2" -m 46
Client mqtt-mareevanastya-d10vxo sending CONNECT (0)
Client mqtt-mareevanastya-d10vxo received CONNACK (0)
Client mqtt-mareevanastya-d10vxo sending PUBLISH (00, q0, r0, m1, "/devices/wb-msw-v3_64/controls/CO2", ... (2 bytes))
Client mqtt-mareevanastya-d10vxo sending DISCONNECT
user@fireboard-ASV7IXPS:~$ mosquitto_pub -d -h dev.rigtech.io -i mqtt-mareevanastya-d10vxo -t "/devices/wb-msw-v3_64/controls/CO2" -m 46
Client mqtt-mareevanastya-d10vxo sending CONNECT
Client mqtt-mareevanastya-d10vxo received CONNACK (0)
Client mqtt-mareevanastya-d10vxo sending PUBLISH (00, q0, r0, m1, "/devices/wb-msw-v3_64/controls/CO2", ... (2 bytes))
Client mqtt-mareevanastya-d10vxo sending DISCONNECT
user@fireboard-ASV7IXPS:~$ mosquitto_pub -d -h dev.rigtech.io -i mqtt-mareevanastya-d10vxo -t "/devices/wb-msw-v3_64/controls/CO2" -m 52
Client mqtt-mareevanastya-d10vxo sending CONNECT
Client mqtt-mareevanastya-d10vxo received CONNACK (0)
Client mqtt-mareevanastya-d10vxo sending PUBLISH (00, q0, r0, m1, "/devices/wb-msw-v3_64/controls/CO2", ... (2 bytes))
Client mqtt-mareevanastya-d10vxo sending DISCONNECT
user@fireboard-ASV7IXPS:~$ mosquitto_pub -d -h dev.rigtech.io -i mqtt-mareevanastya-d10vxo -t "/devices/wb-msw-v3_64/controls/Illuminance" -m 34
Client mqtt-mareevanastya-d10vxo sending CONNECT
Client mqtt-mareevanastya-d10vxo received CONNACK (0)
Client mqtt-mareevanastya-d10vxo sending PUBLISH (00, q0, r0, m1, "/devices/wb-msw-v3_64/controls/Illuminance", ... (2 bytes))
Client mqtt-mareevanastya-d10vxo sending DISCONNECT
user@fireboard-ASV7IXPS:~$
```

Рисунок 2.8 – Отправка данных

Перейдем в ранее созданном устройстве во вкладку журнал. Здесь можно ознакомиться с полученными данными. Данные получены успешно, а значит отладка не требуется.

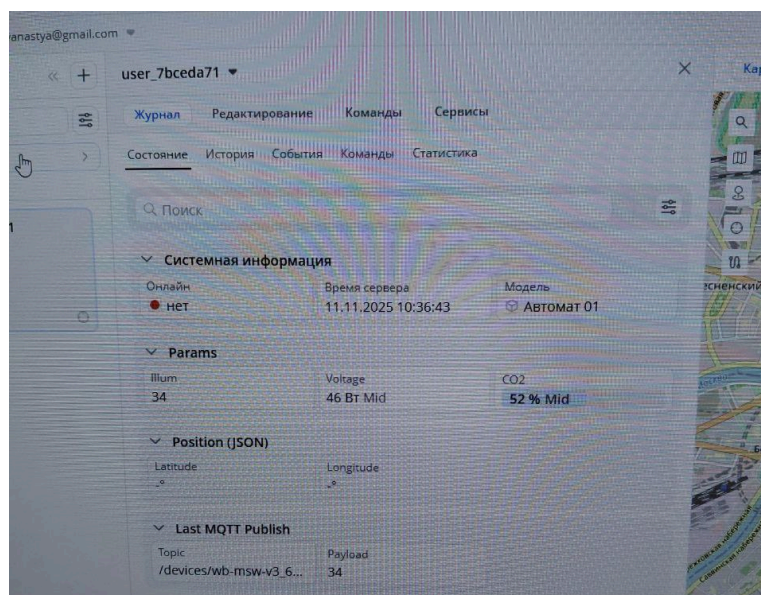


Рисунок 2.9 – Полученные данные

2.4 Заключение

В ходе работы подключили и настроили устройство с поддержкой MQTT на платформе Rightech IoT Cloud, используя внешний MQTT-брокер. В отчете отражены созданные устройства, процесс отправки данных в облако, а также отображение этих данных на виртуальных устройствах в облачной платформе.

ПРАКТИЧЕСКАЯ РАБОТА №10

3.1 Цель работы

Цель раздела «Логика» заключается в повышении уровня абстракции: от работы с моделями и элементами устройства к созданию сценариев автоматического взаимодействия с ним. Каждый сценарий автоматизации — это алгоритм, который определяет логику поведения объекта. С помощью такого сценария можно отслеживать изменения в устройстве и, в зависимости от этих изменений, автоматически выполнять соответствующие действия без участия пользователя. Сценарий автоматизации основан на конечном автомате, состоящем из состояний и переходов между ними.

Конечный автомат (или детерминированный конечный автомат, ДКА) — это математическая модель, используемая для описания поведения систем с конечным числом состояний. Конечный автомат состоит из множества состояний, событий и правил, которые определяют, как система переходит из одного состояния в другое на основе событий или входных данных.

3.2 Постановка задачи

На платформе Rightech реализовать сценарии, предложенные в таблице 1 согласно своему варианту, используя приведенную в методичке теоретическую информацию.

В отчете необходимо отразить файл конфигурации для подключения, схему автомата, описать логику работы автомата, скриншоты состояний и переходов, состояния устройств в журнале объектов.

Таблица 3.1 - индивидуальный вариант

№ варианта	Сценарии
5	Включение и выключение вентилятора по датчику движения. Включение и выключения любого индикатора комбинированного датчика по кнопке

3.3 Ход работы

Создание сценария автоматизации

Для подключения облачной платформы Rightech к демонстрационному стенду Wirenboard (чемодану) необходимо в домашней директории пользователя добавить конфигурацию в файл mosquitto.conf(листинг 3.1).

Листинг 3.1 - файл mosquitto.conf

```
#rightech connect config
connection rightech
address dev.rightech.io:1883
clientid mqtt-mareevanastya-keha8s
try_private false
start_type automatic
topic /devices/wb-mdm3/controls/K2 both
topic /devices/wb-msw-v4_46/controls/Current\Motion both
```

Для создания нового автомата перейдем на вкладку «Логика» и добавим новый автомат, нажав на «+». Укажем имя и модель автомата. После нажатия кнопки «Создать» откроется новый автомат с несколькими элементами в рабочей области. Добавим сюда необходимые элементы для создания сценария согласно номеру варианта(рис. 3.1).

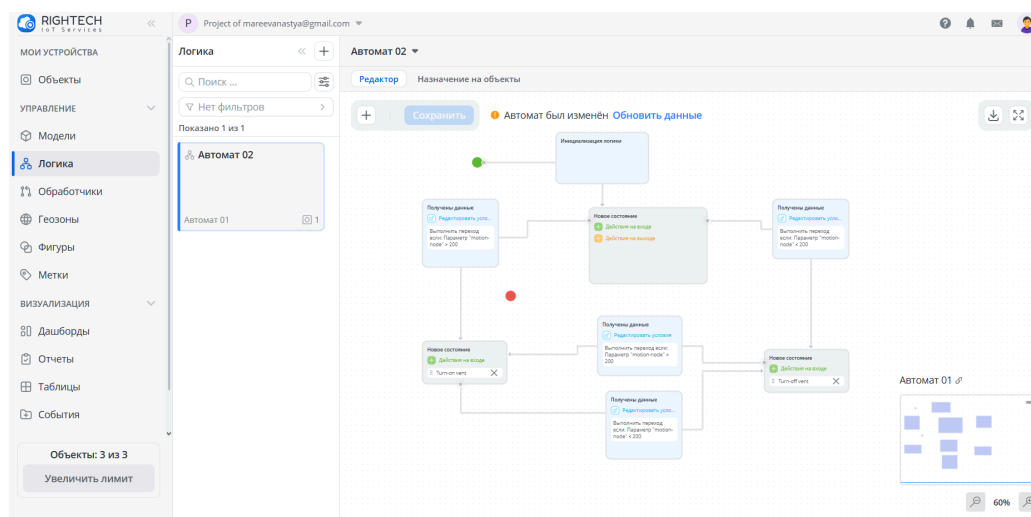


Рисунок 3.1 — Элементы рабочей области нового автомата

Рассмотрим основные элементы рабочей области автомата:

1. Зеленый круг – является обозначением начала исполнения сценария (используется в любом автомате).
2. Инициализация логики – состояние инициализации сценария (присутствует в любом автомате).
3. Новое состояние – в данном элементе указываются действия, которые должны быть выполнены на входе и выходе.
4. Добавить событие (Переход) – В переходе задается событие (всегда) и условие (опционально), при которых происходит изменение состояний.
5. Красный круг – является обозначением окончания исполнения сценария (данный элемент может не использоваться, если автомат предполагает циклическое выполнение без остановки).

Добавим новое состояние(рис. 3.2) и заполним данными. При его добавлении автоматически добавится «Переход».

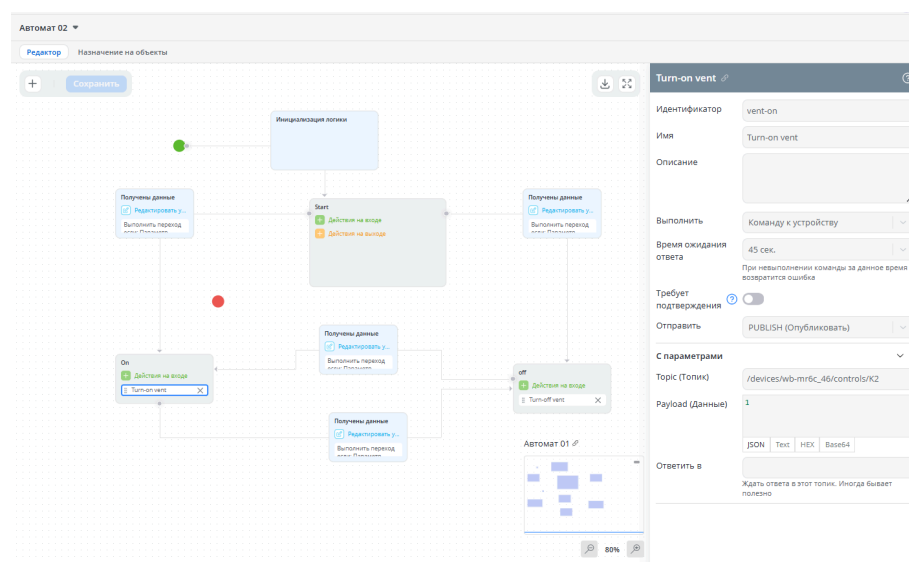


Рисунок 3.2 – Новое состояние

Аналогично логике описанной в практике 9 заполним аргумент button-node и motion-node(рис. 3.3-3.4).

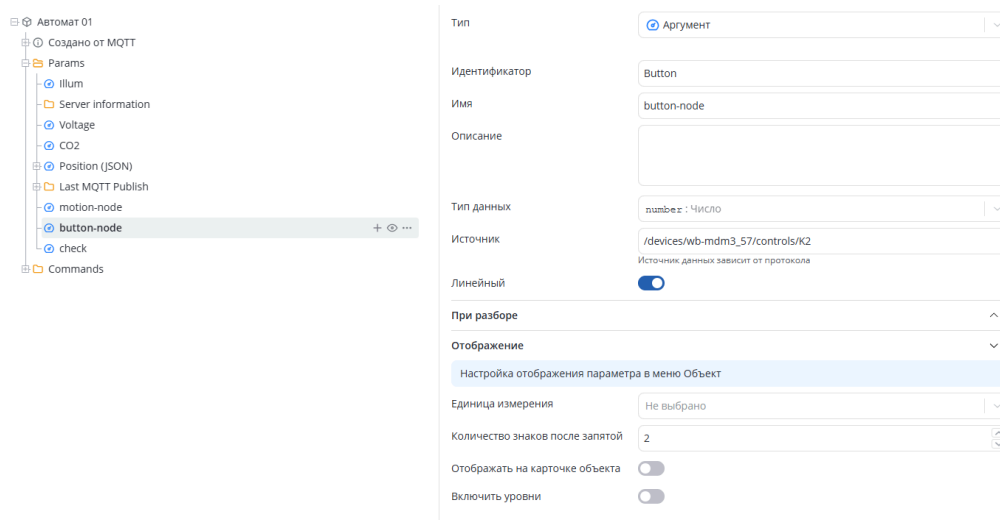


Рисунок 3.3 – Аргумент button-node

+ Создать новый объект с этой моделью

Сохранить

Тип	Аргумент
Идентификатор	Motion
Имя	motion-node
Описание	
Тип данных	number : Число
Источник	/devices/wb-msw-v4_64/controls/Current Motion
	Источник данных зависит от протокола
Линейный	☒

При разборе

Отображение

Настройка отображения параметра в меню Объект

Единица измерения	Не выбрано
Количество знаков после запятой	2
Отображать на карточке объекта	☐
Включить уровни	☐

Рисунок 3.4 – Аргумент motion-node

В параметрах карточек «События» (Переход) запишем следующие данные (рис. 3.5):

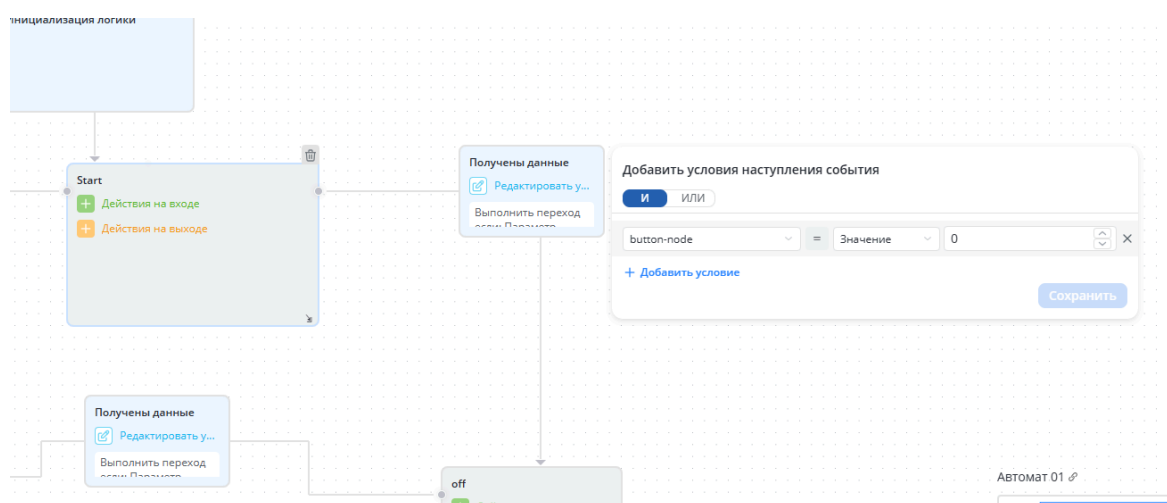


Рисунок 3.5 – Карточка «События»

Теперь запустим наш автомат (рис. 3.6) для первого сценария: включение и выключение вентилятора по датчику движения.

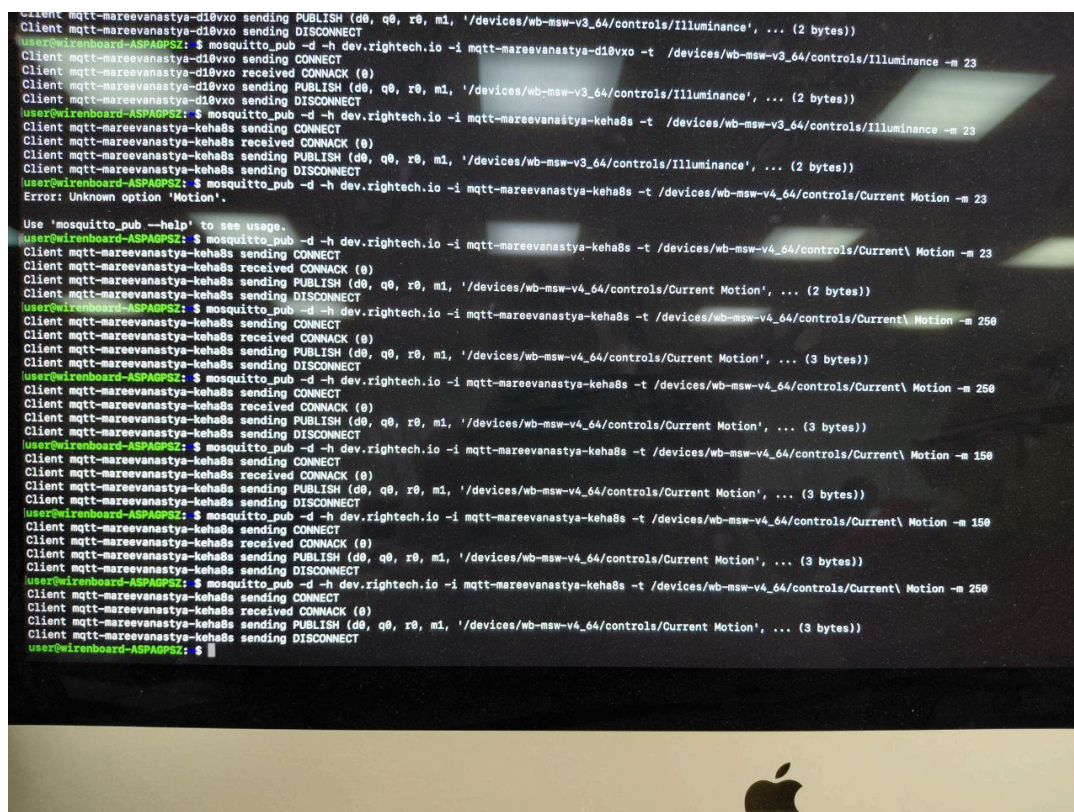


Рисунок 3.6 –запуск автомата

Теперь перейдем на вкладку «Объекты» и ознакомимся с результатами: его статус изменился, а значит логика работы верная.

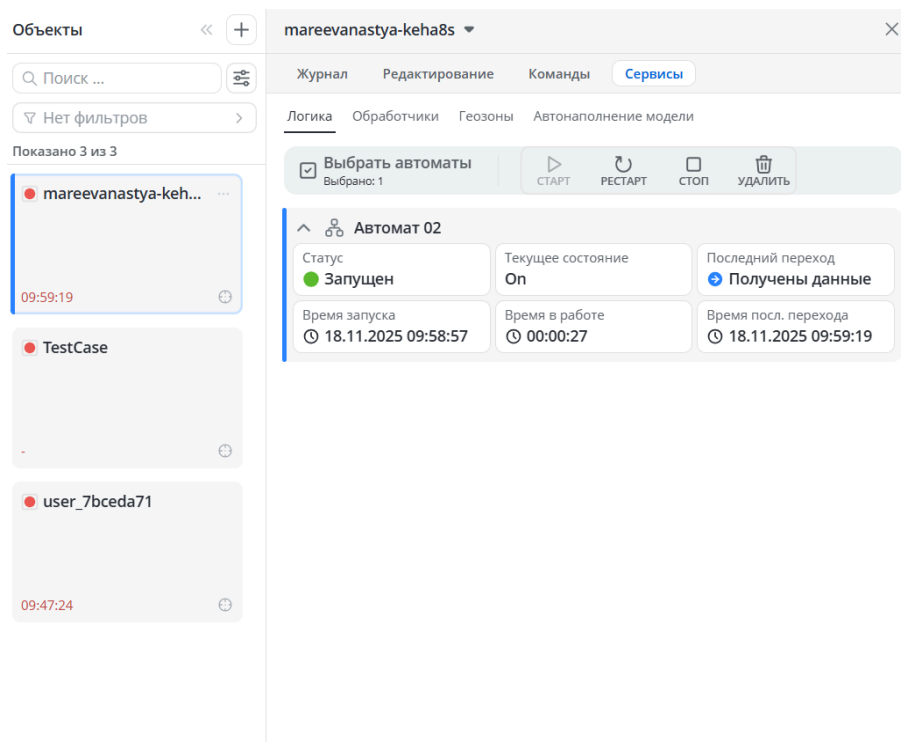


Рисунок 3.7 – Полученные данные

Теперь запустим наш автомат (рис. 3.5) для второго сценария: включение

и выключение любого индикатора комбинированного датчика по кнопке(рис. 3.8)

```
user@wrenboard-ASPA6PSZ: $ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v4_64/controls/Current\ Motion
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d0, q0, r0, m1, '/devices/wb-msw-v4_64/controls/Current Motion', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wrenboard-ASPA6PSZ: $ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v4_64/controls/Current\ Motion
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d0, q0, r0, m1, '/devices/wb-msw-v4_64/controls/Current Motion', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wrenboard-ASPA6PSZ: $ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v4_64/controls/Current\ Motion
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d0, q0, r0, m1, '/devices/wb-msw-v4_64/controls/Current Motion', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wrenboard-ASPA6PSZ: $ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-mdm3_57/controls/K2 -m 1
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d0, q0, r0, m1, '/devices/wb-mdm3_57/controls/K2', ... (1 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wrenboard-ASPA6PSZ: $ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-mdm3_57/controls/K2 -m 1
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d0, q0, r0, m1, '/devices/wb-mdm3_57/controls/K2', ... (1 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wrenboard-ASPA6PSZ: $ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-mdm3_57/controls/K2 -m 0
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d0, q0, r0, m1, '/devices/wb-mdm3_57/controls/K2', ... (1 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wrenboard-ASPA6PSZ: $
```

Рисунок 3.8 – Запуск автомата

Снова перейдем на вкладку «Объекты» и ознакомимся с результатами: его статус изменился, а значит логика работы верная (рис. 3.9).

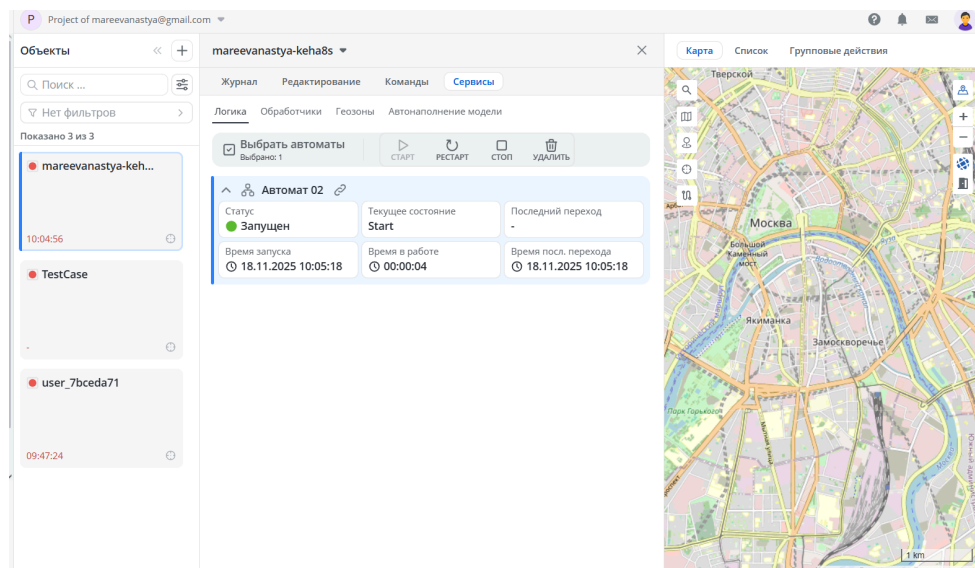


Рисунок 3.9 – Полученные данные

Обратим внимание что после запуска автомата также срабатывает и вентилятор на демонстрационном стенде(рис. 3.9), согласно логике автомата,

что еще раз подтверждает правильность выполненной работы.



Рисунок 3.10 – Пример работы вентилятор

3.4 Заключение

На платформе Rightech реализовали сценарии, предложенные в таблице 1 согласно своему варианту, используя приведенную в методичке теоретическую

информацию.

Подготовили файл конфигурации для подключения, схему автомата, описали логику работы автомата, отразили в отчете скриншоты состояний и переходов, состояния устройств в журнале объектов.

ПРАКТИЧЕСКАЯ РАБОТА №11

4.1 Цель работы

Добавить в логику правил из 10 практической работы формирование нескольких типов тревог.

4.2 Постановка задачи

Добавить в логику правил из 10 практической работы формирование нескольких типов тревог:

- Для первого объекта логики из варианта – тревогу при выходе приходящего параметра за допустимые границы (границы задать самостоятельно и проверьте работоспособность на физическом датчике);
- Для второго объекта логики из варианта – тревогу при отсутствии ожидаемого параметра в приходящем сообщении (к примеру, в приходящем сообщении отсутствует параметр с состоянием кнопки);

В отчет включить обновленные цепочки правил, скрипты проверок и формирования тревог, а также результаты тестирования цепочек при помощи утилит mosquito.

4.3 Ход работы

Реализуем вывод сообщения при отсутствии ожидаемого параметра . Для этого необходимо установить дополнительный блок перехода, проверяющий соответствие параметров, в которые мы установим(рис. 4.1).

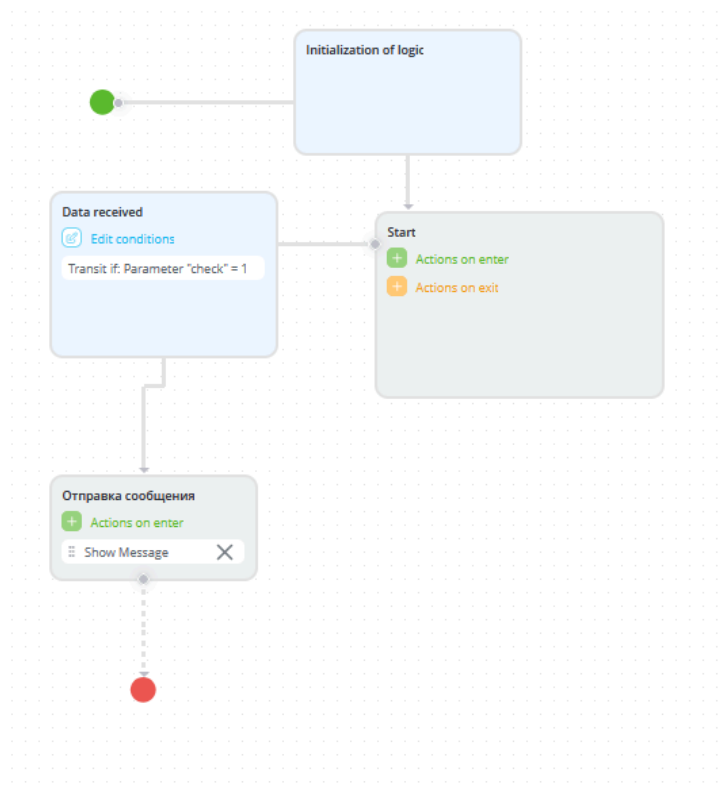


Рисунок 4.1 – Обновленный автомат

Для проверки соответствия входящих данных, создадим в модели дополнительный параметр флаг `check`. Далее перейдем в раздел обработки и создадим новый обработчик. Присвоим нашему обработчику объект с данными, которого он будет взаимодействовать (рис. 4.2).

MSG ▾

Edit [Handler management](#)

[Assignment to objects](#)
[Assignment of parameters](#)

☒ Select objects
Selected: 1

START STOP REMOVE

Selected: 1 / 1 [Select all](#)

#	Object	Model	Status	Last packet time	Link
1	mareevanastya-keha8s	Автомат 01	Processing	-	

Рисунок 4.2 – Присвоили обработчику объект с данными

В разделе параметры добавим ссылки на входные и выходные параметры(рис. 4.3).

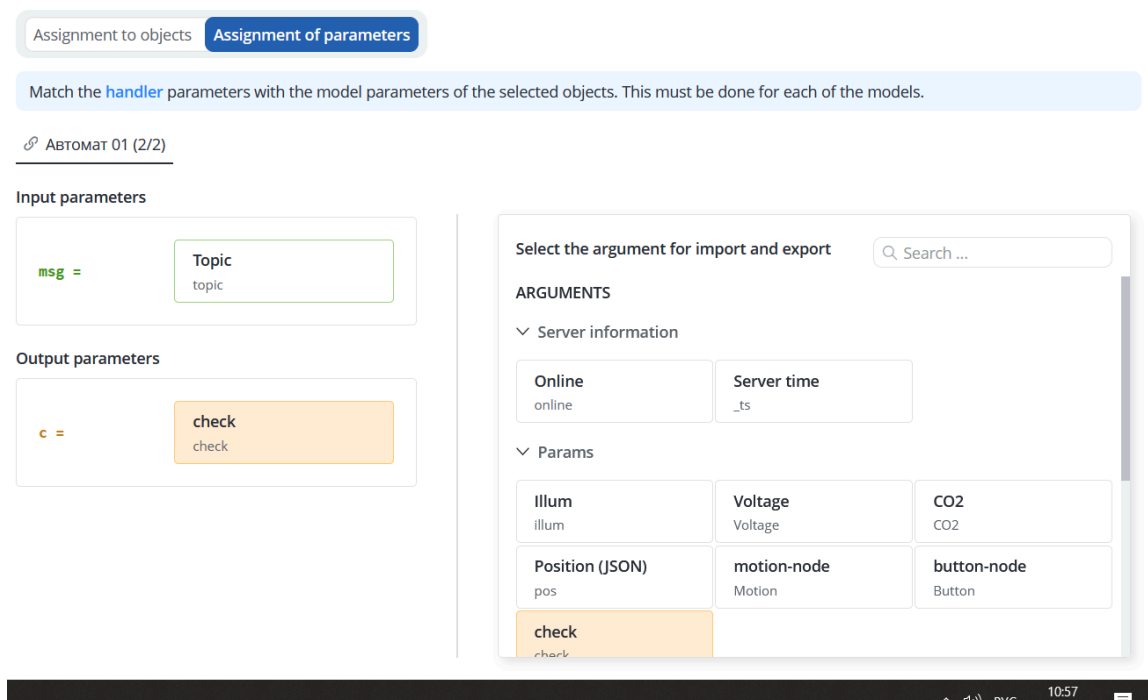


Рисунок 4.3 – Добавим ссылки

При создании нового обработчика перед нами откроется готовый шаблон с небольшим кодом для расчета суммы двух входных значений. Используем его в качестве наглядного примера того, каким образом может быть реализован обработчик и изменим под свой вариант (рис. 4.4).

Сохраним изменения обработчика и рассмотрим из чего он состоит

1. Комментарии.
2. В круглых скобках указаны имена входных параметров.
3. Далее код обработчика, работающий с входными параметрами по заданному алгоритму.
4. В фигурных скобках после слова `return` указываются имена выходных параметров.

```

1  /** Docs and examples:
2      *
3      * - https://rightech.io/developers/handlers
4      * - https://github.com/rightech/handler-lib
5      */
6
7  /**
8      * Makes sum of two numbers
9      * @author `mareevanastya@gmail.com`
10     */
11     function process(msg) {
12         var c;
13
14         if(msg != '/devices/wb-mdm3_57/controls/K2'){
15             c = 1;
16         }else{
17             c = 0;
18         }
19         return {c};
20     }

```

Рисунок 4.4 – Обработчик

Аналогично повторим для сценария с превышением заданного параметра. Теперь убедимся что сообщения доставлены(рис. 4.5-4.6).

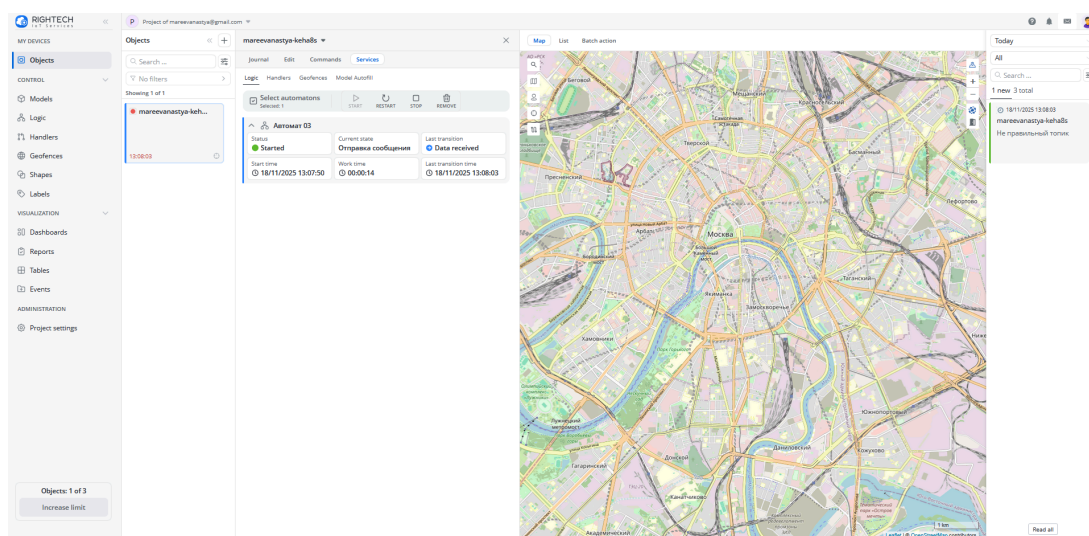


Рисунок 4.5 – Сообщение доставлено

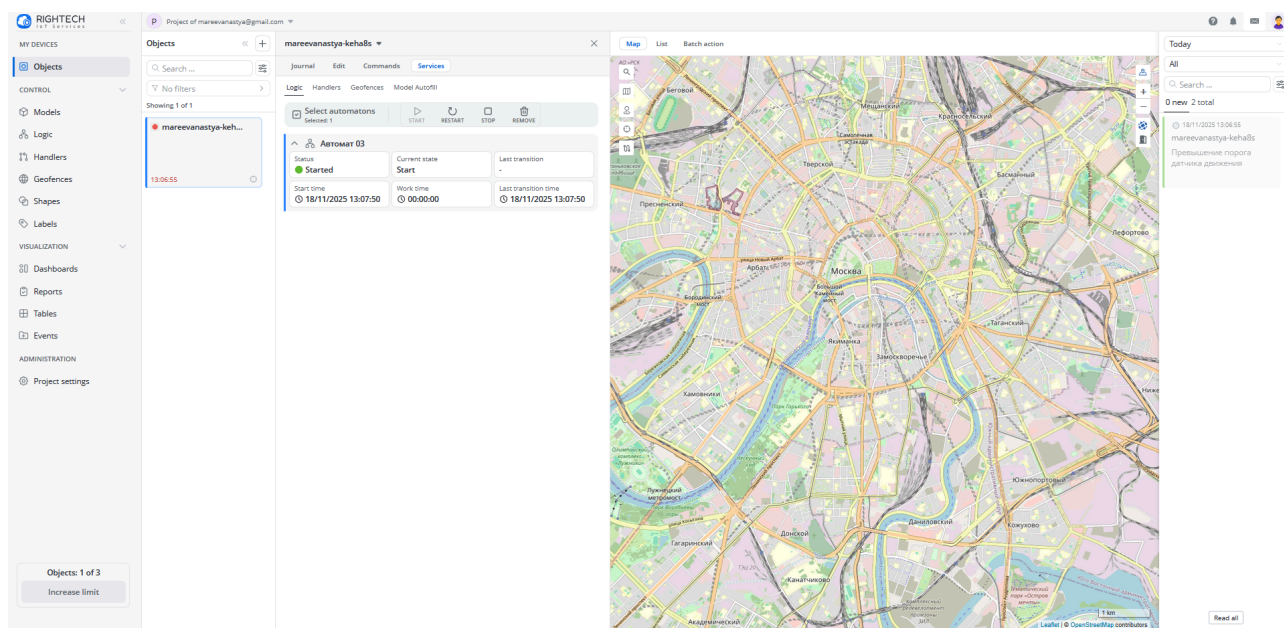


Рисунок 4.6 – Сообщение доставлено

ПРАКТИЧЕСКАЯ РАБОТА №12

5.1 Цель работы

Реализовать отправку e-mail сообщений из облачной платформы используя SMTP.

5.2 Постановка задачи

Реализовать отправку e-mail сообщений из облачной платформы при возникновении тревог на автомате, созданных в практической работе №11. В качестве SMTP сервера для пересылки сообщений предлагается использовать Yandex и Google.

В отчет включить обновленную схему автомата, параметры настройки SMTP-сервера и отправки письма на электронную почту, а также результаты тестирования SMTP-сервера, автомата и скриншоты приходящих электронных писем

5.3 Ход работы

в данной практической предстоит работа с SMTP. Это простой протокол передачи почты. С английского языка переводится, как Simple Mail Transfer Protocol. SMTP сервер отвечает за отправку почтовых рассылок. Его задача, как правило, состоит из двух основных функций:

проверка правильности настроек и выдача разрешения компьютеру, который пытается отправить электронное сообщение;

отправка исходящего сообщения на указанный адрес и подтверждение успешной отправки сообщения. Если доставка невозможна, сервер возвращает отправителю ответ с ошибкой отправки.

Перед началом работы необходимо определить адреса отправителя, получателя и наличие содержимого письма – это обязательные условия, без которых письмо не будет отправлено.

Перейдем к настройке SMTP-сервера Прежде чем отправить оповещение

с помощью электронных писем, необходимо настроить SMTP-сервер. Для этого в меню перейдите в раздел Администрирование – Настройки проекта и выберем вкладку Интеграции (рис. 5.1).

SMTP is a communication protocol for electronic mail transmission. Add an SMTP-server to be able to send email from [Automatons](#)

Message channel name *

Description

Host *

Port *

Username *

Password *

Sender

Connection security

Testing

Note that testing is performed for the saved version of SMTP.

Recipient *

Email Subject *

Email body (text) *

Test email was sent successfully

Рисунок 5.1 – Настройка SMTP-сервера

Обновим схему автомата так чтобы при возникновении тревоги отправлялось письмо на почту (рис. 5.2).

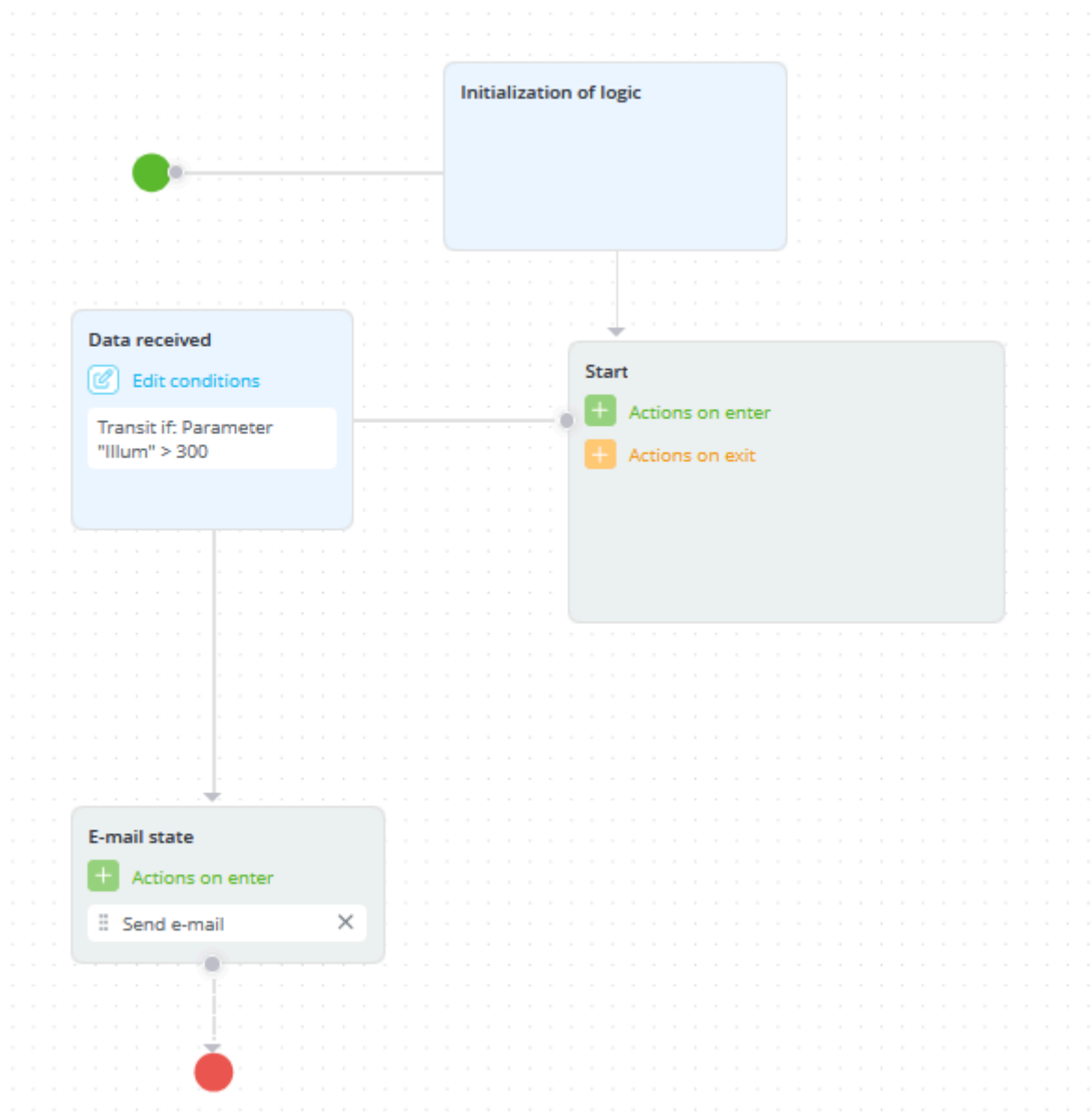


Рисунок 5.2 – Обновленный автомат

Отправим данные соответствующие условию для вызова тревоги на автомате, чтобы получить письмо на почту (рис. 5.3).


```
Welcome to Wiren Board 6.5.2 (s/n AL2MSDPH), release wb-2507 (as stable)
Linux wirenboard-AL2MSDPH 6.8.0-wb140 #1 SMP Fri Jul 18 00:00:00 UTC 2025 arch64 GNU/Linux

System load:  2.02 0.59 0.20   Up time:      0 min
Memory usage: 11% of 1.94G   Usage of /:   45% of 2.00   /mnt/data:  15% of 130

13 package updates are available; type 'apt update && apt upgrade' to update them.

Last login: Tue Nov 18 06:19:55 2025 from 192.168.2.99
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Illuminance -m 36
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Illuminance', ... (2 bytes))
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Illuminance -m 36
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Illuminance', ... (2 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Current\ Motion -m 400
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Current Motion', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Current\ Motion -m 400
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Current Motion', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Illuminance -m 400
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Illuminance', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Illuminance -m 400
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Illuminance', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Illuminance -m 400
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Illuminance', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Illuminance -m 400
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Illuminance', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$ mosquitto_pub -d -h dev.rightech.io -i mqtt-mareevanastya-keha8s -t /devices/wb-msw-v3_64/controls/Illuminance -m 400
Client mqtt-mareevanastya-keha8s sending CONNECT
Client mqtt-mareevanastya-keha8s received CONNACK (0)
Client mqtt-mareevanastya-keha8s sending PUBLISH (d9, q0, r0, m1, '/devices/wb-msw-v3_64/controls/Illuminance', ... (3 bytes))
Client mqtt-mareevanastya-keha8s sending DISCONNECT
user@wirenboard-AL2MSDPH:~$
```

Рисунок 5.3 – Отправка данных

Проверим что данные отправились в разделе статистика нашего объекта(рис. 5.4).

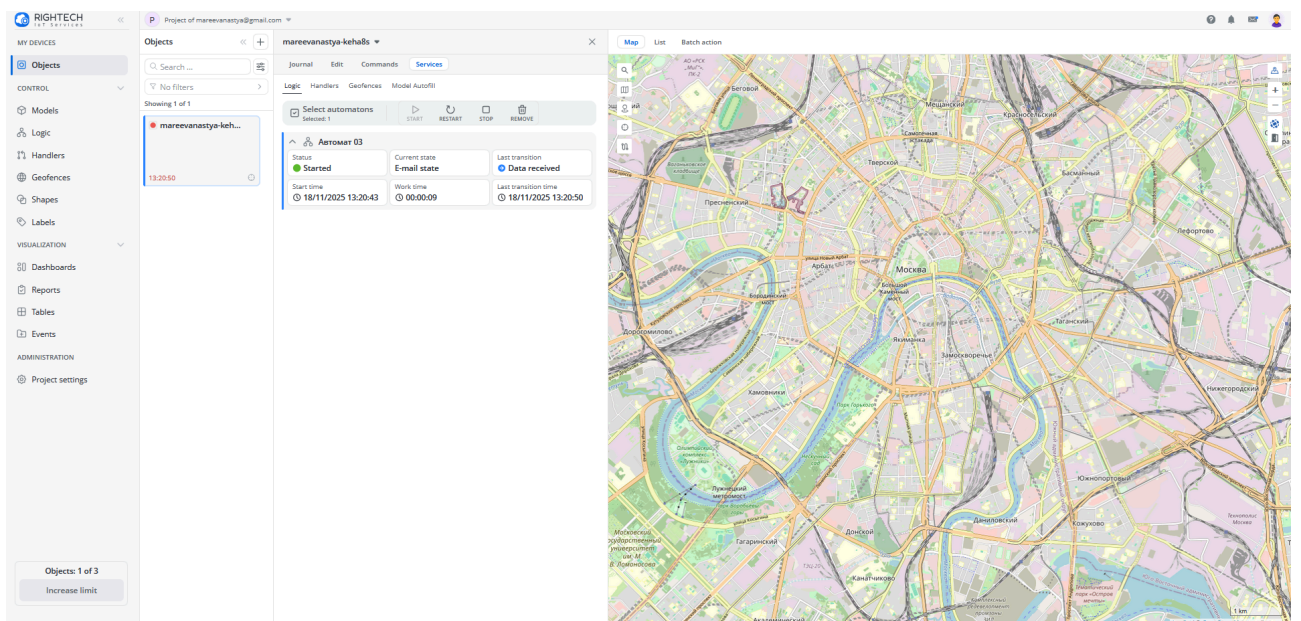


Рисунок 5.4 – Проверка корректности отправки данных

На почту пришло письмо уведомляющее о превышении значения датчика. Письмо получено, а значит практическая работа выполнена верно(рис. 5.5)

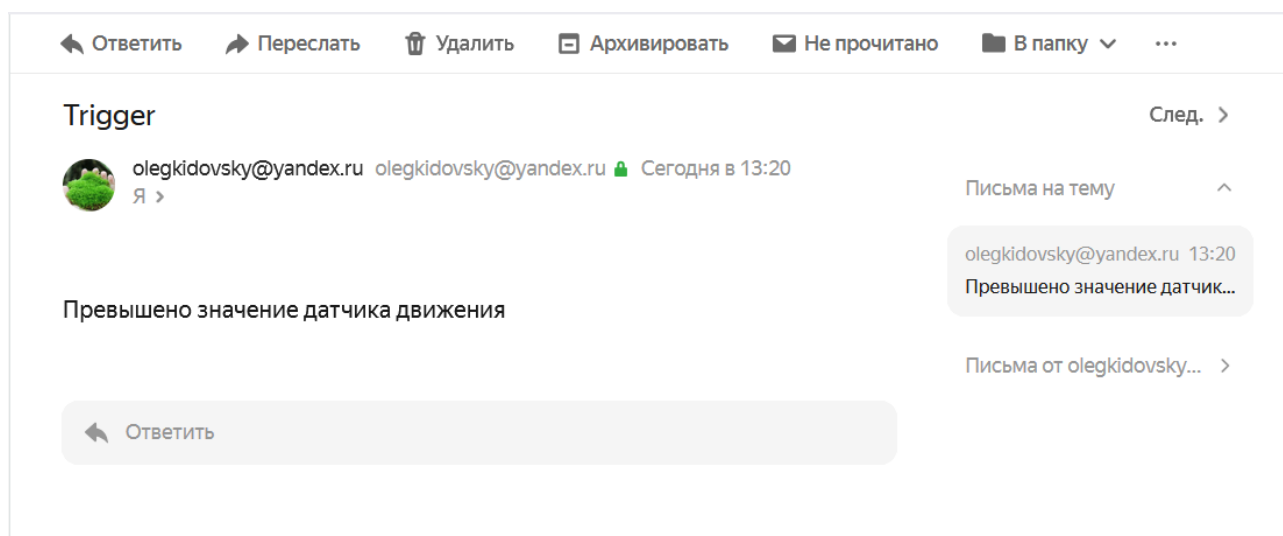


Рисунок 5.5 – Сообщение получено на почту

Повторим описанную процедуру, чтобы также получить сообщение о неверном топике (рис. 5.6-5.8).

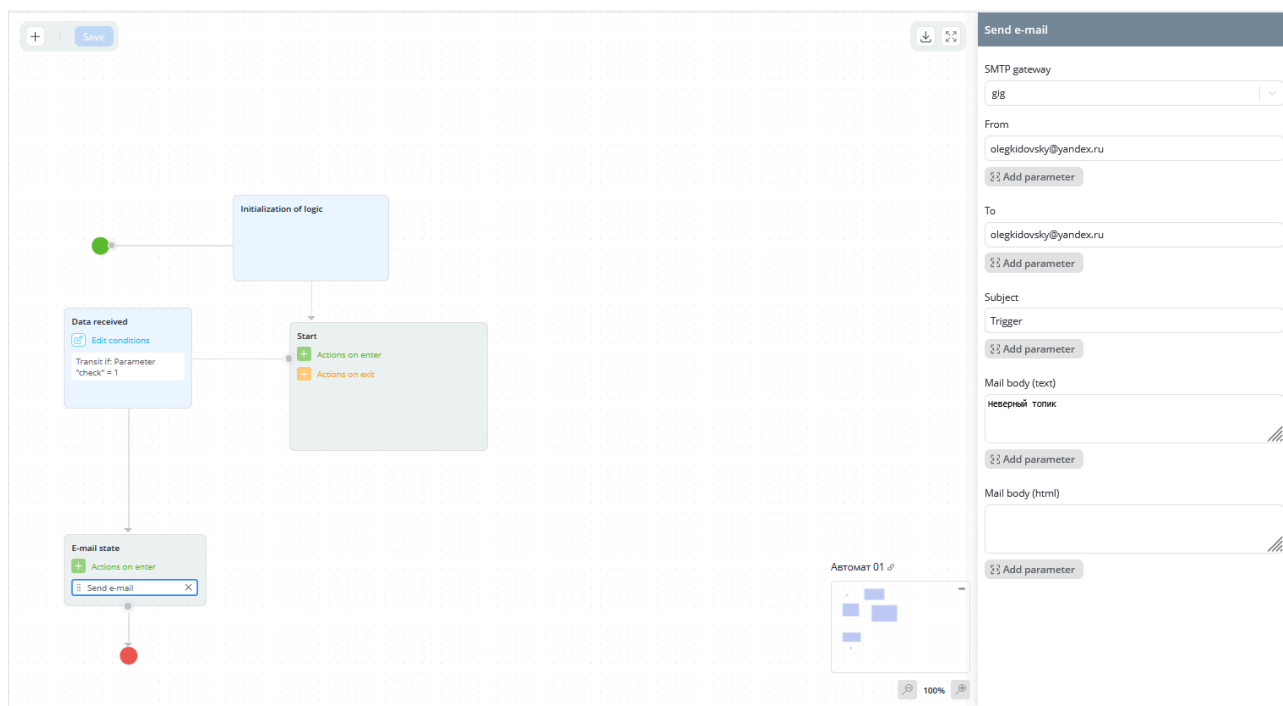


Рисунок 5.6 – Обновленная схема

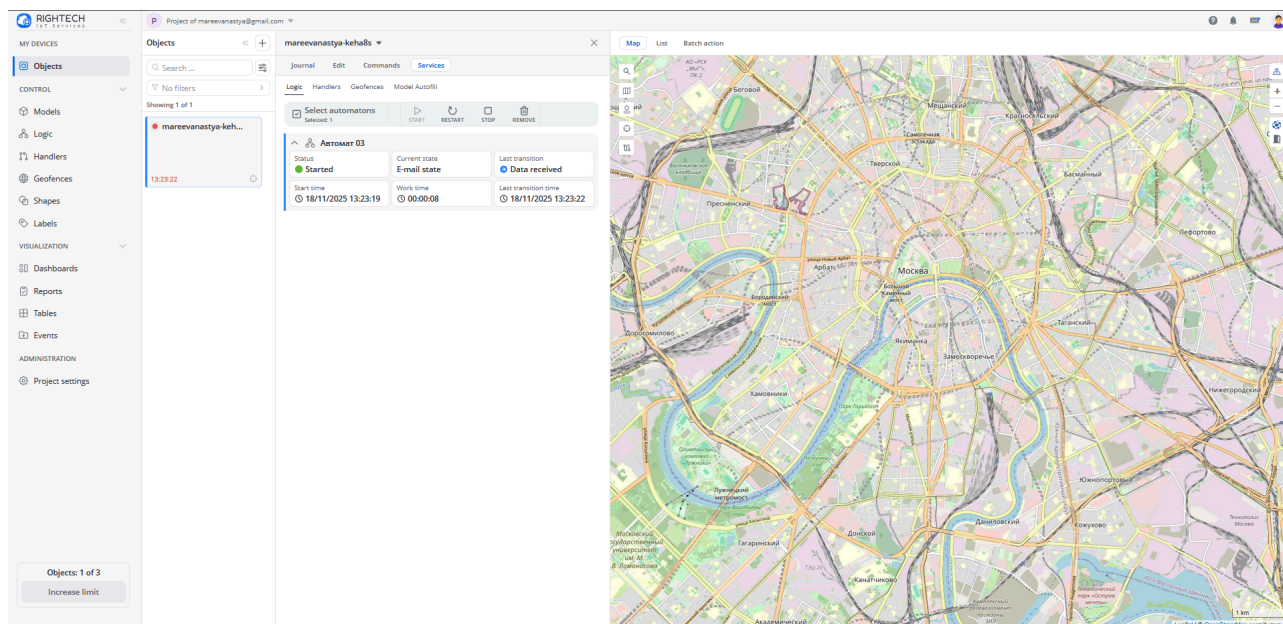


Рисунок 5.7 – Проверка корректности отправки данных

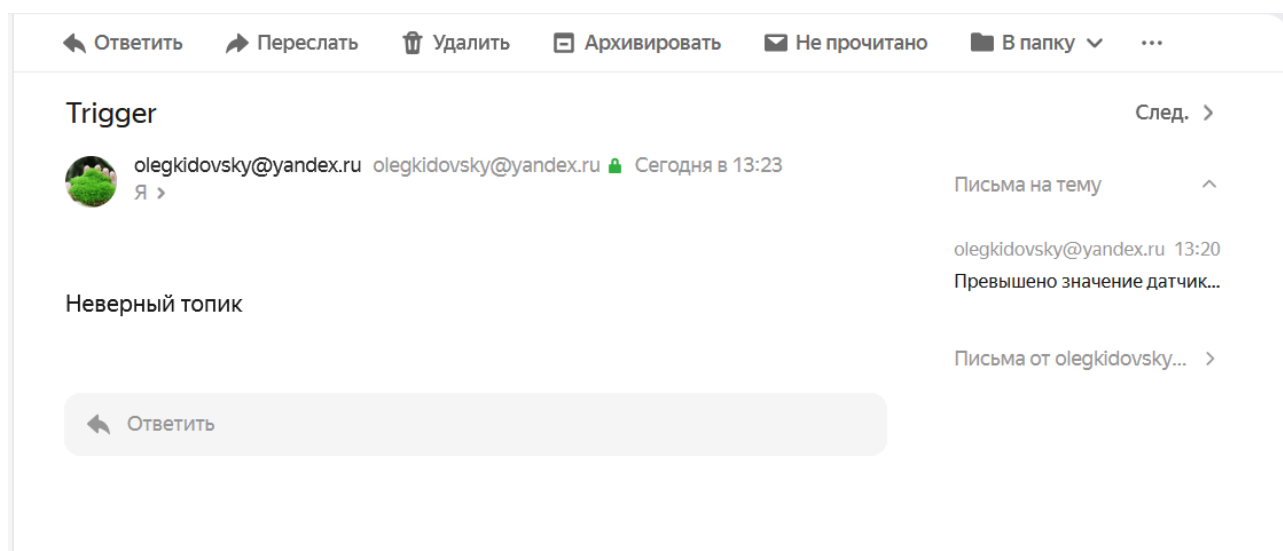


Рисунок 5.8 – Сообщение получено на почту

5.4 Заключение

Реализовали отправку e-mail сообщений из облачной платформы используя SMTP.

ЗАКЛЮЧЕНИЕ

На основании проделанной работы можно сделать вывод об успешном освоении ключевых принципов построения IoT-решений на платформе Rightech IoT Cloud. В ходе выполнения практических заданий были поэтапно реализованы основные компоненты типичного IoT-проекта: от создания моделей устройств и настройки MQTT-подключения до разработки сложной логики автоматизации с использованием конечных автоматов. Особое внимание было уделено практическому применению механизмов мониторинга и оповещения, что позволило создать полноценную систему, способную не только собирать и обрабатывать данные с датчиков, но и автоматически реагировать на критические события.

Важным результатом работы стала успешная интеграция различных сервисов платформы. Настройка сценариев автоматизации для управления устройствами на основе данных с датчиков движения и кнопок, реализация системы тревог для контроля выхода параметров за допустимые границы и отслеживания целостности данных, а также настройка SMTP-рассылки для оперативного оповещения о возникающих инцидентах продемонстрировали гибкость и мощь платформы Rightech для построения комплексных IoT-решений. Все сценарии были протестированы с использованием физического оборудования и программных эмуляторов, что подтвердило их работоспособность и корректность реализации.

Таким образом, в рамках цикла практических работ были получены и закреплены навыки, необходимые для разработки, развертывания и сопровождения систем Интернета вещей. Освоенные методики позволяют создавать отказоустойчивые и масштабируемые решения, обеспечивающие сбор телеметрии, автоматическое управление подключенными устройствами и своевременное оповещение пользователей о событиях, что соответствует современным требованиям к промышленным и исследовательским IoT-проектам.

СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1. Wiren Board. Документация по контроллерам и модулям [Электронный ресурс]. – Режим доступа: <https://wirenboard.com/wiki/> (дата обращения: 23.09.2025).
2. MQTT. Протокол обмена сообщениями в системах Интернета вещей [Электронный ресурс]. – Режим доступа: <http://mqtt.org> (дата обращения: 23.09.2025).
3. Кравцов, А. А. Системы Интернета вещей: архитектура, протоколы, безопасность / А. А. Кравцов. – М.: Горячая линия – Телеком, 2021. – 312 с.
4. Гольдштейн, Б. С. Введение в Интернет вещей: учебное пособие. – СПб.: Питер, 2020. – 256 с.